

Grand Canonical Integration on Loch

Porting the GCI titration pipeline from `grand` to `loch`

1 Overview

What GCI is for. The GCMC/MD pipeline tells us *where* waters sit in a binding pocket. Grand Canonical Integration (GCI) answers a different question: *how favourable* is it for them to be there. It does this by running many independent simulations of the same pocket, each at a different water chemical potential, recording the average occupancy $\langle N \rangle$ in each, and integrating the resulting titration curve to obtain a binding free energy for the water network (Ross *et al.*, JACS 2015).

What changed. The original implementation used the `grand` library (CPU). The rest of the project has moved to `loch` (GPU). This work ports GCI to `loch`, keeping the physics identical while fixing three structural weaknesses of the original that would otherwise have been carried across.

What was delivered. Three new scripts plus three small edits to the shared Loch helper module:

File	Lines	Role
<code>gci_common.py</code>	743	The $B \leftrightarrow \mu$ algebra, construction of the dummy atom that anchors the sphere, seven validators, and the titration record writer.
<code>gci_window.py</code>	810	Runs one titration window. Same structure as <code>ev71_production.py</code> : parse, checkpoint, run, publish.
<code>gci_map_centre.py</code>	329	Carries a hydration-site coordinate from one equilibrated frame into another, and checks it is usable there.
<code>run_gci_sweep.sh</code>	141	Runs the ladder of windows. Deliberately hard-coded and relative-path only.
<code>gci_analyse.py</code>	560	Pools the windows and integrates them into a free energy.
<code>ev71_loch_common.py</code>	(edited)	Three changes, each defaulting to the previous behaviour.

One invocation is one window, exactly as before.

Vocabulary

GCMC sphere

The region where waters are inserted and deleted. For GCI it is centred on a specific hydration site, *not* on the ligand.

Adams value B

The dimensionless knob that sets how favourable insertion is, $B = \beta\mu' + \ln(V_{\text{GCMC}}/V^\circ)$. Sweeping B is the titration.

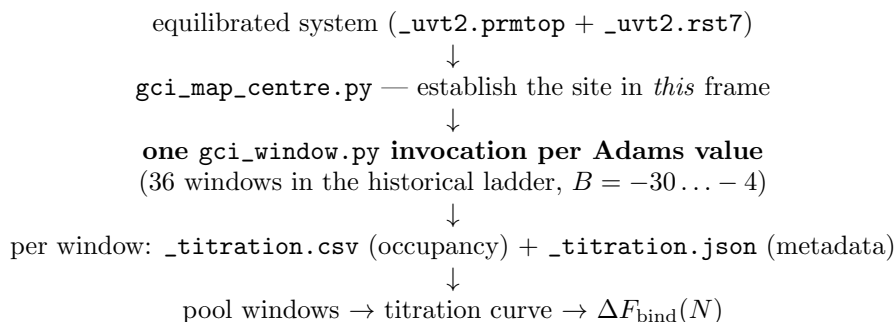
Ghost water

A water present in the topology with its charge and van der Waals interaction switched off. Inserting a water means switching one on; the atom count never changes. The pool of ghosts caps how many waters can be inserted.

μVT

Constant chemical potential, volume, temperature. GCI runs here: no barostat.

2 Overall structure



Inside a single window the flow is:

1. resolve the (B, μ) pair for the radius actually requested;
2. load the equilibrated system, audit its waters and ligand;
3. insert a massless dummy atom at the sphere centre;
4. run six preflight validations, all of which raise on failure;
5. build the Loch sampler at this window's μ , and *verify the B it actually got*;
6. 625 cycles of {4000 MD steps, then 400 GCMC attempts, then record occupancy};
7. finalise the topology, audit the water arithmetic, publish a checkpoint.

3 The central obstacle: Loch cannot be told where the sphere is

In ordinary GCMC/MD the sphere follows the ligand, which Loch supports directly through a selection such as `"resname LIG"`. GCI needs something else: the sphere must sit on an *arbitrary fixed point*, a hydration site identified by clustering crystallographic waters, which no atom selection reproduces.

Loch offers no way to say this. Its `reference` argument must be a string, and there is no coordinate parameter anywhere in the API. Worse:

Loch silently ignores unknown arguments

`GCMCSampler.__init__` ends in `**kwargs` and reads only two keys from it. So

```
GCMCSampler(..., sphere_centre=[47.1, 42.2, 43.1])
```

constructs successfully, ignores the coordinates entirely, and reports no error. A plausible-looking run would sample around the ligand instead of the requested site.

The original solved this with a trick that survives the port: put a **massless dummy atom** in the topology at the desired point and select it by name. Loch recomputes the sphere centre every move as the centre of geometry of the reference atoms; with one atom of zero mass, OpenMM never integrates it, and μ VT has no barostat to rescale it, so the centre is genuinely fixed.

How the dummy is built — then and now

The original did this with text surgery on a PDB plus a one-atom OpenMM force field:

original: Production_titration.py

```
dum_line = (f"HETATM99999  SPH DUM A{dum_resid:>4}      "
            f"{x:>8.3f}{y:>8.3f}{z:>8.3f}  1.00  0.00      \n")
```

```
# ... inserted before the final END, plus dum.xml declaring type SPH (mass 0,
# charge 0, sigma 0.1 nm, epsilon 0), plus forcing atom.element = None so the
# elementless template matches, plus zeroing mass/charge/epsilon again by hand.
```

The Loch branch is AMBER/Sire-based, not PDB/XML-based, so the dummy is constructed as a proper one-atom molecule and appended to the system:

```
new: gci_common.build_dummy_molecule
cursor["charge"] = AtomCharges(info, 0.0 * legacy_units.mod_electron)
cursor["LJ"]      = AtomLJs(info, LJParameter(1.0 * legacy_units.angstrom,
                                              0.0 * legacy_units.kcal_per_mol))
cursor["mass"]    = AtomMasses(info, 0.0 * legacy_units.g_per_mol)
cursor["element"] = AtomElements(info, Element(0)) # "Xx"; == grand's element = None
cursor["coordinates"] = atom_coordinates
cursor["forcefield"] = forcefield # copied from a water molecule
cursor["connectivity"] = Connectivity(info)
cursor["intrascala"] = CLJNBPairs(info, CLJScaleFactor(0, 0))
```

Verified equivalent

The resulting OpenMM particle is **bit-identical** to what `dum.xml` produced: mass 0 Da, charge 0 e , $\sigma = 0.1$ nm, $\epsilon = 0$ kJ/mol. Measured drift of the sphere centre over a run: 2.8×10^{-14} Å. Rejected alternatives were `tleap` (cannot append one atom to an existing topology without rebuilding it) and `ParmEd` (requires hand-plumbing a new nonbonded type).

Two details worth noting. The selection is `"resname DUM and atomname SPH"` — deliberately *without* a residue number, because an AMBER topology stores residue labels but not residue numbers, so a resid-based selection would not survive being written and reloaded. And because Loch appends its ghost waters at the *end* of the system, the dummy's atom index is stable; the code asserts this rather than assuming it.

4 Three weaknesses fixed in the port

1. The chemical potential was calibrated to one sphere size

B and μ are related through the sphere volume:

$$B = \frac{\mu'}{k_B T} + \ln \frac{V_{\text{GCMC}}}{V^\circ}, \quad V_{\text{GCMC}} = \frac{4}{3} \pi r^3.$$

The original swept μ from a hardcoded list of 36 values. Those values were computed for a 4 Å sphere — but the project used *two* spheres (7 Å and 4 Å, see §5), and the study report states the same B ladder was used for both. Applying the 4 Å list to the 7 Å sphere silently shifts every window:

$$\mu = -19.1837 \text{ kcal/mol} \quad \text{gives} \quad B = -30.00 \text{ at } r = 4 \text{ Å} \quad \text{but} \quad B = -28.32 \text{ at } r = 7 \text{ Å}$$

The port removes the possibility. You pass `--target-b`; μ is derived from the radius you actually gave:

```
def mu_from_target_b(target_b, *, radius_angstrom, standard_volume_angstrom3,
                    adams_shift=0.0, temperature_K=TEMPERATURE_K):
    volume_ratio = sphere_volume_angstrom3(radius_angstrom) / standard_volume_angstrom3
    return kt_kcal_per_mol(temperature_K) * (
        target_b - adams_shift - math.log(volume_ratio))
```

`kt_kcal_per_mol` is Loch's own β expression inverted, so our B and Loch's agree to machine precision. Passing `--mu` directly is still supported; the implied B is computed and recorded either way.

And then it is checked against Loch. Because of the silent-`**kwargs` hazard, deriving μ correctly is not enough — we must confirm the sampler received it. Loch never exposes B publicly, but it stores $\exp(B)$:

```
def assert_loch_adams_matches(sampler, expected_b, *, tolerance=1.0e-9):
    exp_b = float(getattr(sampler, "_exp_B"))
    actual = math.log(exp_b)
    if abs(actual - float(expected_b)) > float(tolerance):
        raise RuntimeError(...) # radius, V0, mu or adams_shift did not reach the sampler
```

This single assertion catches a mistyped keyword, a radius/ μ mismatch, and a wrong standard volume. Observed in practice: requesting $B = -6.0$ yields `adams_value_from_loch = -6.000000000000234`.

2. The titration observable was never written to a file

In the original, the occupancy $\langle N \rangle$ existed only inside prose log lines:

```
400 move(s) completed (3 accepted (0.7500 %)). Current N = 0. Average N = 0.445
```

and the analysis notebook recovered it by regular-expression parsing of *scheduler stdout*. There was no N -versus- B output file at all.

The port writes an explicit, all-numeric record, so the standard pipeline validator `finite_csv` can verify the checkpoint schedule exactly:

```
step,cycle,md_steps_completed,sphere_waters,accepted_moves,accepted_attempts,insertions,deletions
,ghost_pool
400,1,200,1,2,12,1,1,27
800,2,400,1,4,25,2,2,27
...
4000,10,2000,2,24,161,13,11,25
```

Why the occupancy must be read with an explicit context

`sampler.num_waters()` called with no argument returns a value cached at the start of the last GCMC batch. It is blind to waters that diffused across the sphere boundary during the intervening MD — which, in GCMC/MD, is a real flow of water. Passing the context forces a fresh recount on the GPU:

```
sphere_waters = int(sampler.num_waters(context))
```

Calling it before `print_sampler` also makes the printed value truthful for free.

Alongside the CSV, each window writes a `_titration.json` recording B , the B Loch confirmed, μ , the radius, V° , kT , the equilibrium B , the sphere centre and its realised position, the ghost pool minimum, and `attempts_per_cycle`. That last one matters: it is the checkpoint interval in GCMC attempts, and the original analysis notebook had it hardcoded to the wrong value (200 instead of 400), silently averaging over twice the intended window. Recording it removes the assumption.

3. Nothing tied the sphere centre to the structure being simulated

This is the subtlest failure and the one worth understanding, because every other check on the sphere is *self*-consistent: the dummy is exactly where it was asked to be, it is frozen, it is a single atom, Loch resolves it. None of that can tell whether the coordinate means anything in *this* input.

A hydration-site coordinate belongs to the frame it was measured in. Every equilibration run produces its own frame. Reusing a centre across frames puts the sphere somewhere arbitrary — and the run completes cleanly, publishes a valid checkpoint, and yields a titration curve for the wrong region. The study report describes exactly this outcome for its apo system: the regions “*primarily sampled bulk solvent rather than the hydration sites of interest*”, and those results had to be discarded.

Measured, for the same coordinate in two different equilibrated frames of the same protein:

Frame	Nearest solute atom	Solute in sphere	Waters in sphere
the frame the centre came from	3.05 Å	8	2 (a real pocket)
a different equilibration	1.08 Å	20	0 (buried)

At 1.08 Å the centre is physically *inside* a heavy atom; no water can ever occupy it. The port therefore adds a guard that rejects both impossible cases — buried in solute, or out in bulk solvent with no pocket nearby:

```
environment = validate_sphere_environment(
    system, reference, radius,
    min_solute_clearance_angstrom=opt.min_solute_clearance, # default 2.0 Å
    max_solute_distance_angstrom=opt.max_solute_distance,   # default radius + 4 Å
    require_waters_in_sphere=opt.require_waters_in_sphere)
```

It reports what it found, and warns (without failing) if the sphere starts dry:

```
Sphere environment: nearest solute heavy atom 3.14 Å; inside the sphere
5 solute heavy atoms and 1 water oxygens
```

The distance search runs inside Sire rather than over every atom in Python, which took the check from over two minutes to 0.27 s.

Carrying a site into a new frame: `gci_map_centre.py`

Rejecting a stale centre is necessary but not sufficient — you still need the right one. Since the historical coordinates are the only record of where these two sites are, the helper transfers them: it superposes the protein Cα atoms of the target structure onto the reference structure and applies the same rigid transform to each named site.

```
python -u gci_map_centre.py \
    --reference-structure GCI/CRY1AN139uvt2.pdb \
    --target-prmtop <yours>_uvt2.prmtop --target-rst7 <yours>_uvt2.rst7 \
    --site Sphere1 55.900 48.000 44.140 7.0 \
    --site Sphere2 47.132 42.204 43.126 4.0 \
    --output mapped_sites.json
```

It then runs the *same* environment guard `gci_window.py` will apply, so an unusable centre is caught here rather than after 36 jobs have been submitted, and prints the arguments to paste onward:

```
Superposed 476 atoms ('name CA and protein'): RMSD 1.25 Å
Sphere1: [55.9, 48.0, 44.14] -> [52.8815, 51.8824, 52.0925] (r = 7 Å)
```

```
PASS nearest solute heavy atom 2.64 Å; inside the sphere 36 solute heavy atoms and 5 water
oxygens
Sphere2: [47.132, 42.204, 43.126] -> [44.6842, 46.7018, 47.9135] (r = 4 Å)
PASS nearest solute heavy atom 2.86 Å; inside the sphere 9 solute heavy atoms and 0 water
oxygens
```

Ready-to-run centres:

```
# Sphere1
--sphere-centre 52.8815 51.8824 52.0925 --sphere-radius 7 --num-ghosts 144
# Sphere2
--sphere-centre 44.6842 46.7018 47.9135 --sphere-radius 4 --num-ghosts 27
```

Three points of care. The superposition is rejected above a configurable C α RMSD (default 3 Å), because a poor fit means the structures are not comparable and a transferred coordinate would be meaningless. The Kabsch solution explicitly corrects for an improper rotation, which would otherwise mirror the structure and place the site on the wrong side of the pocket. And atom correspondence is established by comparing the ordered sequence of residue *names*, not their numbers:

Why names and not numbers

Residue numbering is a file-format convention — a PDB is typically 1-based while the same protein read from an AMBER topology is 0-based. Comparing numbers rejected a perfectly good alignment (GLY1 versus GLY0, 476 times). Worse, the offset is not even constant here, because modelled loops break a simple shift; the JSON records it as `null` when non-uniform. The ordered name sequence is what actually establishes that the two selections describe the same chain in the same order.

The JSON record carries the input hashes, the rotation and translation, the RMSD, and the environment audit for every site, so a mapped centre is reproducible and attributable rather than a number someone once pasted into a shell.

Note that Sphere 2 maps to a genuine pocket (2.86 Å clearance, 9 solute heavy atoms) that happens to contain no water in that particular frame. That passes, with the warning: an empty pocket is a legitimate starting state, and the titration exists precisely to find out how favourable filling it is.

5 Two spheres, not one ambiguity

The original script on disk specified $r = 7$ Å at one centre; the data that was published came from $r = 4$ Å at a different centre. This looked at first like the file had been left in an inconsistent state. The study report settles it:

“two spherical regions were defined, a larger one encompassing 7 experimental water molecules, Sphere 1, and a smaller one encompassing the remaining two, and respectively having radius of 7 and 4 Å.”

Both are legitimate. The script was edited in place to switch between them, and the numbers in it are simply whichever was configured last.

	Radius	Centre (original frame)	Crystal waters	Reported N^*
Sphere 1	7 Å	[55.900, 48.000, 44.140]	7	25
Sphere 2	4 Å	[47.132, 42.204, 43.126]	2	2

Consequently `--sphere-radius` and `--sphere-centre` are **required arguments with no defaults**. Both enter the checkpoint signature, so a window can never be confused with one run on a different region, and the choice is always explicit and recorded.

6 Changes to shared code

Three edits to `ev71_loch_common.py`, each defaulting to the existing behaviour. All six existing call sites were verified to receive byte-identical arguments.

<code>make_sampler</code>	Gained seven keyword-only arguments (<code>reference</code> , <code>radius</code> , <code>excess_chemical_potential</code> , <code>standard_volume</code> , <code>num_ghost_waters</code> , <code>adams_shift</code> , <code>bulk_sampling_probability</code>), each defaulting to the value it previously hardcoded. Also validates them up front, where a failure is cheap and attributable, rather than letting Loch raise after cloning the whole system.
<code>validate_gcmc_handoff</code>	Gained <code>num_ghost_waters</code> . Latent bug fixed: it hardcoded 45, so the expected-water arithmetic was silently wrong for any other buffer size — which GCI always uses.
<code>save_system</code>	Latent bug fixed: used <code>Path.with_suffix</code> , which replaces everything after the last dot. A prefix such as <code>run_mu-8.45</code> became <code>run_mu-8</code> , so two windows could overwrite each other’s topologies. Since Sire cannot write <i>any</i> basename containing an extra dot, the fix is to reject such a prefix with an actionable message instead of failing deep inside the writer.

One intentional consequence

`implementation_signature` hashes the *source* of these modules, so editing them changes every existing checkpoint signature. Completed stages will re-run if the pipeline is invoked again. Their outputs remain independently verifiable via `validate_recorded_outputs`, which is hash-only. This is the provenance design working as intended, and it was an explicit decision rather than a side effect.

7 What is unchanged

The physics is the original’s. Verified against the study report’s methods table:

	Original	Port
Temperature / friction / timestep	300 K / 1 ps ⁻¹ / 2 fs	identical
Electrostatics / cutoff / switch	PME / 12 Å / 10 Å	identical
Constraints	H-bonds	identical
Standard volume V°	30.345 Å ³	identical
$C\alpha$ restraints	100 kJ mol ⁻¹ nm ⁻² , plus the dummy	identical
Ensemble	μ VT, no barostat	identical
Schedule	625 × (400 attempts + 8 ps MD)	identical
Total per window	5.0 ns, 250 000 attempts	identical

The schedule deserves a footnote, because the copy of the script on disk carried a much shorter one (8 ps total, with an unreachable branch) that was clearly a leftover test. Three independent sources agree on the real schedule: the study report’s methods text, the headers of the archived output files, and the spacing of the log records in them (exactly 400 moves apart, 625 records, 5.0 ns).

8 Safety properties

The repository convention is to fail loudly rather than proceed, and GCI follows it. Each window performs, in order: an input water and ligand audit; six preflight validations of the dummy and the sphere; the Adams cross-check against Loch; a water-count audit of the sampler topology; a save-and-reload audit proving the dummy survived the AMBER round trip; a live OpenMM check that the particle is frozen; a realised-centre check; a per-cycle guard on the ghost pool; and a final water-arithmetic audit. Every one raises.

The convention proved itself during development

At one point the paths the script declared and the paths `save_system` wrote diverged. Rather than publishing a marker pointing at files that did not exist, `complete_checkpoint` refused to write it and the run failed. The bug surfaced immediately instead of becoming a silently incomplete dataset.

9 Running a sweep

`run_gci_sweep.sh` loops over the ladder, one window at a time. It is intentionally hardcoded: for a stage whose failure modes are still being learned, a script you can read top to bottom beats a configurable one. Everything is relative to the working directory, so there is no environment to get wrong.

```
run_gci_sweep.sh
gci_window.py      gci_common.py
ev71_loch_common.py pipeline_utils.py
CRY1KL101uvt2.prmtop CRY1KL101uvt2.rst7      <- your two inputs
```

Everything the sweep exposes lives in one block at the top:

```
PREFIX="CRY1KL101"          # no dots: Sire reads the format from the extension
PRMTOP="CRY1KL101uvt2.prmtop"
RST7="CRY1KL101uvt2.rst7"
CENTRE_X="48.331"; CENTRE_Y="57.025"; CENTRE_Z="56.001"
RADIUS="4.0"
NUM_GHOSTS="27"
RUN_ROOT="gci_runs"
SMOKE=1          # 1 = ~1 min/window to check the setup; 0 = the real 5 ns schedule
```

then a plain `for` loop over the 36 hardcoded B values, each into its own directory. Run it with `./run_gci_sweep.sh`.

Three choices in it are worth stating, because each one is about being able to debug the thing later:

- **Preflight before the loop.** All six required files are checked once, so a missing input fails immediately instead of 36 times.
- **One bad window does not abort the sweep.** The exit status is recovered from `PIPESTATUS`, since piping through `tee` for the per-window log would otherwise mask it. Failures are collected and reported in the summary, and the script exits non-zero.
- **No skip logic in the shell.** Re-run handling is left entirely to `gci_window.py`'s own checkpoint, which re-validates every output hash and returns. Two mechanisms doing one job is what makes a script hard to trust; a window that *failed* has no checkpoint and is simply redone, so re-running after a fix is the natural recovery.

It closes with the titration points, which doubles as a physics check — occupancy must rise with B :


```

===== SUMMARY =====
windows attempted : 2
checkpoints found : 2
all windows complete

Titration points (tail-mean occupancy is the observable):
  B =    -17.0    final sphere_waters = 0
  B =     -6.0    final sphere_waters = 6

```

For this sphere $B_{\text{equil}} = -8.04$, so $B = -6$ should be filled and $B = -17$ empty. Two points already say the sampler is responding in the right direction and at roughly the right place.

Sequential by design, for now

36 windows $\times$ 5 ns is roughly a week on one GPU. The loop is the right tool for understanding the stage and for a handful of windows; the same loop body drops into a SLURM array (`--array 0-35`, `\$SLURM_ARRAY_TASK_ID` in place of `\$i`) when it is time to go wide. That belongs in a sibling script rather than as a flag on this one.

10 How the pieces connect

Four scripts, run in order. Each consumes the previous one's output; nothing else is shared between them.

```

<system>_uvt2.prmtop + <system>_uvt2.rst7  (from equilibration)
      ↓
gci_map_centre.py  - where is the site in this frame?
      ↓ a validated centre + suggested ghost count
run_gci_sweep.sh  - loops the ladder, calling...
      ↓
gci_window.py     - one window per Adams value
      ↓ gci_runs/window_NN_B*/
gci_analyse.py    - pool, integrate, report
      ↓
ΔFbind(N), N*, figures

```

1. gci_map_centre.py — establish the site

Only needed when the coordinate comes from somewhere other than the structure you are about to run. Superposes protein C α and applies the transform.

```

python gci_map_centre.py \
  --reference-structure <where the coordinate came from>.pdb \
  --target-prmtop <yours>_uvt2.prmtop --target-rst7 <yours>_uvt2.rst7 \
  --site Sphere1 55.900 48.000 44.140 7.0 \
  --site Sphere2 47.132 42.204 43.126 4.0 \
  --output mapped_sites.json

```

```

--site NAME X Y Z R    Repeatable. Radius is per site, since regions differ in size.
--max-rmsd              Reject the superposition above this C $\alpha$  RMSD (default 3 Å).
--min-solute-clearance  Reject a centre buried in solute (default 2 Å).
--max-solute-distance   Reject a centre out in bulk (default radius + 4 Å).
--skip-validation       Map only, do not check. Not recommended.

```

It prints the `--sphere-centre` / `--sphere-radius` / `--num-ghosts` arguments to paste into the next step. If the site was instead derived by clustering your own production waters, it is already in the right frame and this step is unnecessary.

2. gci_window.py — one window

The unit of work. Radius and centre are required; there is no default, because more than one region is legitimate.

```
python gci_window.py \  
  --prmtop <sys>_uvt2.prmtop --rst7 <sys>_uvt2.rst7 \  
  --output-dir gci_runs/window_19_B-17.0000 --prefix CRY1AN139 \  
  --window-index 19 \  
  --sphere-centre 47.132 42.204 43.126 --sphere-radius 4.0 \  
  --num-ghosts 27 --target-b -17.0
```

<code>--target-b or --mu</code>	Exactly one. Whichever you give, the other is derived for the radius in use, and both are recorded.
<code>--sphere-radius, --sphere-centre</code>	Required. Both enter the checkpoint signature.
<code>--num-ghosts</code>	Insertion buffer. Defaults to comfortably above the sphere's bulk-equivalent occupancy; too small clips the high- B tail.
<code>--cycles --attempts --md-steps</code>	Schedule. Defaults are the published $625 \times (400 + 4000) = 5$ ns.
<code>--require-waters-in-sphere</code>	Insist the site is hydrated in the input.
<code>--trajectory-stride</code>	Write every n th frame; a full window is ~ 0.5 GB.
<code>--force</code>	Ignore an existing checkpoint and redo the window.

3. run_gci_sweep.sh — the ladder

Hardcoded on purpose (§9). Edit the block at the top, put the two input files and the four Python scripts in one directory, and run `./run_gci_sweep.sh`. Set `SMOKE=1` first: about a minute per window, enough to prove the setup before committing days.

4. gci_analyse.py — pool and integrate

```
python gci_analyse.py --run-root gci_runs --prefix CRY1AN139 --output-dir analysis
```

<code>--equilibrated-fraction</code>	Portion of each window averaged (default 0.5).
<code>--equilibrated-moves</code>	The same thing in GCMC attempts; converted using the recorded checkpoint interval.
<code>--exclude-window-index</code>	Drop an outlier. Repeatable.
<code>--monotonic-fit isotonic</code>	Opt in to a monotone fit for under-converged data. Default refuses instead.
<code>--max-n</code>	Largest N evaluated. Defaults to $\text{round}(\langle N \rangle \text{ at } B_{\text{equil}}) + 1$.
<code>--mu-hydration</code>	Bulk water μ' ; defaults to the recorded value.

It re-hashes every window's outputs before reading them, requires all windows to be the same experiment at different B , and writes the titration curve, the free-energy table, two figures, a summary JSON and its own checkpoint.

Radius, standard volume, kT and the checkpoint interval all come from the window metadata. Nothing about the simulation is hardcoded in the analysis — which is what makes the next section's comparison possible.

11 Checking the analysis against the original

The same archived CRY1/AN139 data was run through the original notebook and the new pipeline. A small converter turns `grand`'s scheduler output into the window format, so only the analysis differs.

	radius	tail	B_{equil}	N^*	$\Delta F(N^*)$
notebook, as shipped	7 Å (<i>wrong</i>)	125 ckpt	-6.359	3	-11.413
notebook, corrected	4 Å	62 ckpt	-8.038	4	-12.399
new, default	4.000 Å (<i>derived</i>)	312 ckpt	-8.037	<i>refuses</i>	
new, isotonic	4.000 Å (<i>derived</i>)	62 ckpt	-8.037	3	-9.991
<i>published</i>	4 Å	—	-8.038	2	-9.397

Three things fall out. The converter recovers radius 4.000 Å and the $625 \times (400 + 4000)$ schedule from the logs alone, independently confirming the protocol. Correcting the notebook’s two hardcoded constants shifts B_{equil} by 1.68 and changes N^* — an error the new pipeline cannot make, because it reads them. And the monotonicity treatment, which is the main difference between the last two rows, is worth **2.4 kcal/mol** on its own.

That last figure is the case for refusing by default: the curve has 13–15 inversions, `np.interp` returns a wrong B_N on non-monotonic input without complaint, and the original analysis never checked.

12 Open items

- **The sweep is sequential.** A SLURM-array sibling of `run_gci_sweep.sh` is the next step for running the full ladder at scale.
- **Centres must be established per frame.** There is no AMBER topology for the frame the historical data used, so those runs cannot be reproduced input-for-input. `gci_map_centre.py` carries a site across (476 Cα, 1.25 Å RMSD for the two frames tested); re-deriving it by clustering waters is the alternative. Comparison with the published work is therefore at the level of B_{equil} , N^* and ΔF_{bind} , not per-window energies.
- **The original ghost buffers were probably too small.** Sphere 1 has a bulk-equivalent capacity of ~ 47 waters but was given 25 ghosts; the buffer caps insertions, so the high- B tail of the curve may have been clipped, and the integral is sensitive to that tail. `suggested_num_ghosts` now proposes 144 for $r = 7$ Å and 27 for $r = 4$ Å, and a per-cycle guard raises with an actionable message if the pool empties.
- **No mid-window restart.** A window is a single 5 ns job; exceeding a wall-clock limit restarts it.